



CSA0501-DATABASE MANAGEMENT SYSTEMS

LAB EXPERIMENTS

DATE: 10/08/2026

EXPERIMENT 19 -- TRIGGER

AIM:

To implement a **BEFORE UPDATE Trigger in MySQL** to log changes made to the student_cgpa table.

PROCEDURE:

1. Open **MySQL Workbench** and select your database.
2. Check the existing student_cgpa table.
3. Create a new table called student_audit to store the old student details.
4. Create a **BEFORE UPDATE trigger** on the student_cgpa table.
5. The trigger automatically stores the old values in student_audit before any update occurs.
6. Update a student record in the student_cgpa table.
7. Display the student_audit table to verify that the old record was logged.

PROGRAM:

1. CREATE TABLE

```
CREATE TABLE student_cgpa (  
    reg_no VARCHAR(15) PRIMARY KEY,  
    student_name VARCHAR(50) NOT NULL,  
    department VARCHAR(20),  
    no_of_subs_pass INT,  
    no_of_subs_fail INT,  
    cgpa DECIMAL(3,2)  
);
```

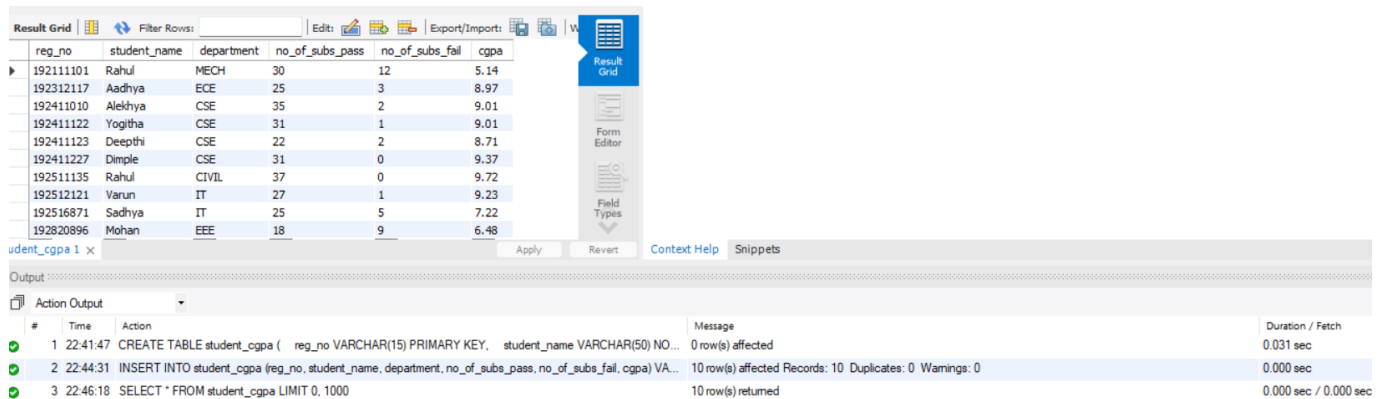
Then Insert Values:

```
INSERT INTO student_cgpa  
(reg_no, student_name, department, no_of_subs_pass, no_of_subs_fail, cgpa)  
VALUES  
(192411123, 'Deepthi', 'CSE', 22, 2, 8.71),  
(192512121, 'Varun', 'IT', 27, 1, 9.23),  
(192312117, 'Aadhya', 'ECE', 25, 3, 8.97),  
(192411227, 'Dimple', 'CSE', 31, 0, 9.37),
```

```
( '192820896','Mohan','EEE',18,9,6.48),
('192411122','Yogitha','CSE',31,1,9.01),
( '192516871','Sadhya','IT',25,5,7.22),
('192111101','Rahul','MECH',30,12,5.14),
('192411010','Alekhyia','CSE',35,2,9.01),
('192511135','Rahul','CIVIL',37,0,9.72);
```

```
SELECT * FROM student_cgpa;
```

OUTPUT:



The screenshot shows a database management tool interface. At the top, there's a 'Result Grid' with columns: reg_no, student_name, department, no_of_subs_pass, no_of_subs_fail, and cgpa. The grid contains 11 rows of student data. Below the grid, there's an 'Output' window showing the execution of three SQL commands. The first command creates the 'student_cgpa' table. The second command inserts 10 rows of data. The third command selects all data from 'student_cgpa' limited to 1000 rows, returning 10 rows.

reg_no	student_name	department	no_of_subs_pass	no_of_subs_fail	cgpa
192111101	Rahul	MECH	30	12	5.14
192312117	Aadhya	ECE	25	3	8.97
192411010	Alekhyia	CSE	35	2	9.01
192411122	Yogitha	CSE	31	1	9.01
192411123	Deepthi	CSE	22	2	8.71
192411227	Dimple	CSE	31	0	9.37
192511135	Rahul	CIVIL	37	0	9.72
192512121	Varun	IT	27	1	9.23
192516871	Sadhya	IT	25	5	7.22
192820896	Mohan	EEE	18	9	6.48

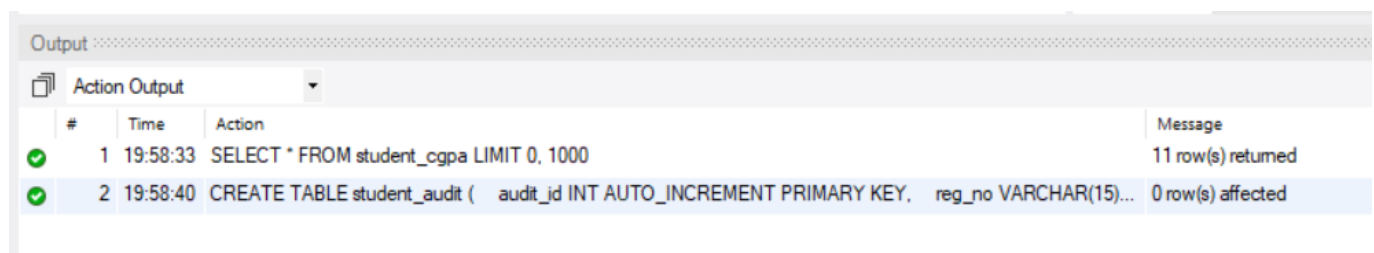
#	Time	Action	Message	Duration / Fetch
1	22:41:47	CREATE TABLE student_cgpa (reg_no VARCHAR(15) PRIMARY KEY, student_name VARCHAR(50) NO...	0 row(s) affected	0.031 sec
2	22:44:31	INSERT INTO student_cgpa (reg_no, student_name, department, no_of_subs_pass, no_of_subs_fail, cgpa) VA...	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0	0.000 sec
3	22:46:18	SELECT * FROM student_cgpa LIMIT 0, 1000	10 row(s) returned	0.000 sec / 0.000 sec

2. Create an Audit Table

This table stores the old details before a student record is updated.

```
CREATE TABLE student_audit (
    audit_id INT AUTO_INCREMENT PRIMARY KEY,
    reg_no VARCHAR(15),
    student_name VARCHAR(50),
    department VARCHAR(20),
    no_of_subs_pass INT,
    no_of_subs_fail INT,
    cgpa DECIMAL(3,2),
    changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

OUTPUT:



The screenshot shows the 'Output' window of a database management tool. It displays the execution of two SQL commands. The first command is a SELECT statement from 'student_cgpa' limited to 1000 rows, returning 11 rows. The second command is a CREATE TABLE statement for 'student_audit', which affects 0 rows.

#	Time	Action	Message
1	19:58:33	SELECT * FROM student_cgpa LIMIT 0, 1000	11 row(s) returned
2	19:58:40	CREATE TABLE student_audit (audit_id INT AUTO_INCREMENT PRIMARY KEY, reg_no VARCHAR(15)...	0 row(s) affected

3. Create the BEFORE UPDATE Trigger

```
DELIMITER $$
```

```
CREATE TRIGGER before_student_update
BEFORE UPDATE ON student_cgpa
FOR EACH ROW
BEGIN
    INSERT INTO student_audit
    (reg_no, student_name, department, no_of_subs_pass, no_of_subs_fail, cgpa)
    VALUES
    (OLD.reg_no, OLD.student_name, OLD.department,
    OLD.no_of_subs_pass, OLD.no_of_subs_fail, OLD.cgpa);
END$$
```

```
DELIMITER ;
```

OUTPUT:

Output				
Action Output				
#	Time	Action	Message	
✓ 1	19:58:33	SELECT * FROM student_cgpa LIMIT 0, 1000	11 row(s) returned	
✓ 2	19:58:40	CREATE TABLE student_audit (audit_id INT AUTO_INCREMENT PRIMARY KEY, reg_no VARCHAR(15)...	0 row(s) affected	
✓ 3	20:00:06	CREATE TRIGGER before_student_update BEFORE UPDATE ON student_cgpa FOR EACH ROW BEGIN I...	0 row(s) affected	

5. Update a Student Record to Test the Trigger

For example, update Deepthi's CGPA:

```
UPDATE student_cgpa
SET cgpa = 9.00
WHERE reg_no = '19241123';
```

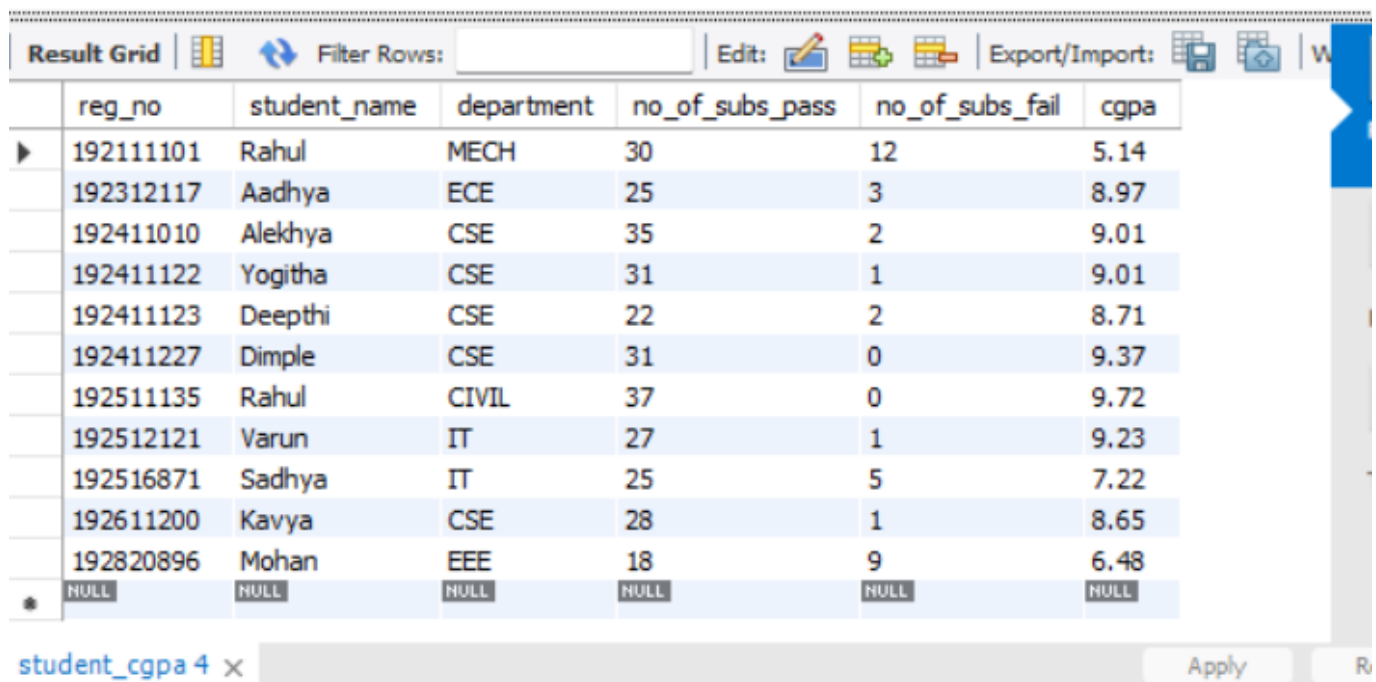
OUTPUT:

Output				
Action Output				
#	Time	Action	Message	
✓ 1	19:58:33	SELECT * FROM student_cgpa LIMIT 0, 1000	11 row(s) returned	
✓ 2	19:58:40	CREATE TABLE student_audit (audit_id INT AUTO_INCREMENT PRIMARY KEY, reg_no VARCHAR(15)...	0 row(s) affected	
✓ 3	20:00:06	CREATE TRIGGER before_student_update BEFORE UPDATE ON student_cgpa FOR EACH ROW BEGIN I...	0 row(s) affected	
✓ 4	20:02:28	UPDATE student_cgpa SET cgpa = 9.00 WHERE reg_no = '19241123'	0 row(s) affected Rows matched: 0 Changed: 0 Warnings: 0	

6. Check the Updated Main Table

```
SELECT * FROM student_cgpa;
```

OUTPUT:



The screenshot shows a database query result grid with the following columns: reg_no, student_name, department, no_of_subs_pass, no_of_subs_fail, and cgpa. The data is as follows:

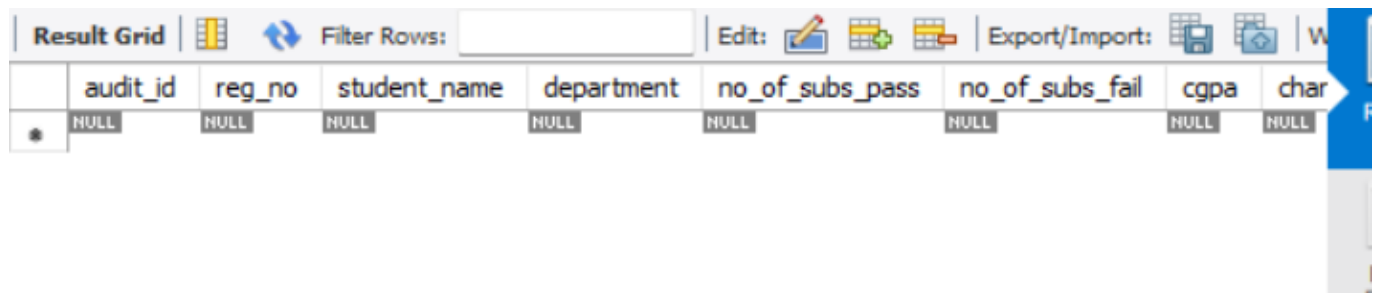
reg_no	student_name	department	no_of_subs_pass	no_of_subs_fail	cgpa
192111101	Rahul	MECH	30	12	5.14
192312117	Aadhya	ECE	25	3	8.97
192411010	Alekhy	CSE	35	2	9.01
192411122	Yogitha	CSE	31	1	9.01
192411123	Deepthi	CSE	22	2	8.71
192411227	Dimple	CSE	31	0	9.37
192511135	Rahul	CIVIL	37	0	9.72
192512121	Varun	IT	27	1	9.23
192516871	Sadhya	IT	25	5	7.22
192611200	Kavya	CSE	28	1	8.65
192820896	Mohan	EEE	18	9	6.48
NULL	NULL	NULL	NULL	NULL	NULL

The interface includes a toolbar with options like 'Filter Rows', 'Edit', and 'Export/Import'. The bottom status bar shows 'student_cgpa 4' and an 'Apply' button.

7. Check the Audit Table

```
SELECT * FROM student_audit;
```

OUTPUT:



The screenshot shows a database query result grid for the student_audit table with the following columns: audit_id, reg_no, student_name, department, no_of_subs_pass, no_of_subs_fail, cgpa, and char. The data is as follows:

audit_id	reg_no	student_name	department	no_of_subs_pass	no_of_subs_fail	cgpa	char
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

The interface includes a toolbar with options like 'Filter Rows', 'Edit', and 'Export/Import'. The bottom status bar shows 'student_audit 4' and an 'Apply' button.

RESULT: Thus the trigger is executed successfully.